TECHNICAL PROJECT REPORT

EEG Seizure Detection

Machine Learning Classification and Verilog RTL Simulation

Submitted by	Krishna Agrawal
Specialization	VACD
Guide / Faculty	Dr. Ananda Y R
Academic year	

Contents

Abstract

1. Introduction

2. System Architecture

3. Design Logic and Methodology

4. Hardware-Oriented Implementation

5. Results and Discussion

6. Timing Analysis

7. Limitations

8. Future Scope

9. Conclusion

References

Abstract

This project looks at detecting seizures from EEG (electroencephalography) data. We trained a Random Forest classifier to separate seizure from non-seizure examples, then wrote the decision logic in Verilog RTL and checked it in simulation with a testbench.

On the validation set the model reached 78.75% accuracy and 76.90% balanced accuracy, with a ROC-AUC of 0.8618 and a PR-AUC of 0.5416. It handles non-seizure examples better than seizure ones. For the seizure class, recall is 0.74 but precision is only 0.35, so the model finds most seizures but also raises quite a few false alarms.

The exported model has 25 trees, a maximum depth of 3, 16 input features and a threshold of about 0.375. On the hardware side we have a synthesized RTL schematic, a simulation waveform, a testbench in which all 20 displayed test cases match their expected outputs, and a power estimate from the synthesis tool.

In short, the model can be evaluated in software and the logic can be simulated in hardware. The work does not show clinical performance, real-time EEG acquisition or operation on a physical FPGA board. Testing on independent recordings and a real hardware implementation would be needed before any practical use.

Keywords: EEG, seizure detection, Random Forest, feature selection, Verilog HDL, RTL, testbench, FPGA, confusion matrix, ROC-AUC, PR-AUC.

1. Introduction

EEG records the electrical activity of the brain, and the signal looks different during a seizure than between seizures. Classifying EEG segments automatically can make analysis more consistent, but the results depend heavily on the data, the preprocessing, the features, how the model is evaluated and where the decision threshold is set.

The project has two parts. First, a Random Forest classifier is trained and evaluated on seizure and non-seizure examples. Second, Verilog RTL and a testbench are used to describe and verify a digital decision path. The numbers in this report come from our project presentation, which holds the model parameters, validation metrics, RTL schematic, waveform, test cases and a power-estimation screenshot.

1.1 Problem statement

The goal is to classify EEG-based input examples as seizure or non-seizure, and to show a matching digital implementation in a hardware description language.

The validation set is imbalanced, with far fewer seizure examples than non-seizure ones:

Class	Examples	Share of set
Non-seizure	1,470	87.2%
Seizure	215	12.8%
Total	1,685	100%

Because of this imbalance, accuracy alone can be misleading. So in this report we look at per-class precision and recall, balanced accuracy, ROC-AUC, PR-AUC and the confusion matrix together.

1.2 Objectives

1. Train and evaluate a Random Forest classifier for seizure versus non-seizure classification.
2. Note down the model configuration: number of trees, maximum depth, threshold and number of selected features.
3. Evaluate the validation predictions using accuracy, balanced accuracy, ROC-AUC, PR-AUC, a classification report and a confusion matrix.
4. Describe the decision logic in Verilog RTL and look at the synthesized schematic.
5. Verify the RTL behaviour with a testbench that has known expected outputs.
6. Go through the power estimate and note the limitations and future work.

2. System Architecture

The project follows two connected paths. The machine-learning path produces a binary prediction from a set of selected input features. The hardware path represents the decision behaviour in RTL and checks it against expected test vectors.

The presentation does not show the whole Random Forest being converted into RTL automatically, so we treat the two paths as related parts of one project rather than a single tool flow. We also do not assume that every tree was built as a hardware decision tree.

2.1 Machine-learning path

Stage	Description
Dataset and labels	Each input example is labelled as seizure or non-seizure. The dataset name, how it was recorded and the preprocessing steps are not given in the presentation.
Feature selection	16 input features are used. Their names are not shown.
Model training	Random Forest with 25 trees and a maximum depth of 3.
Decision stage	A classification threshold of about 0.375.
Validation	Predictions are checked using standard classification metrics and a confusion matrix.

2.2 RTL and simulation path

The RTL side has a synthesized logic schematic and a simulation waveform with testbench output. The schematic shows the logic the synthesis tool generated, and the waveform shows how the signals change during simulation.

The testbench compares the expected output with the actual output for a sequence of samples. The screenshot shows 20 checks and ends with the message "Testbench finished". We did not have the RTL source or the full port list to hand, so internal register names and interface timing are not described here.

3. Design Logic and Methodology

3.1 Random Forest classifier

A Random Forest is a group of decision trees whose outputs are combined into one final prediction. Each tree makes a series of decisions based on the input features, and combining many trees gives a steadier result than relying on a single tree.

Our model has 25 trees, each with a maximum depth of 3. Shallow trees keep the number of sequential decisions small, which suits a hardware implementation. The trade-off is that they may miss more complex patterns in the data.

3.2 Feature selection and threshold

The model works with 16 selected features. The presentation does not list their names, so we do not describe them individually.

The threshold of about 0.375 turns the model score into a yes/no decision. Lowering it catches more seizures but gives more false alarms; raising it cuts false alarms but misses more seizures.

In practice the threshold should be picked using validation data and the needs of the application, not just to push accuracy as high as possible.

3.3 Evaluation methodology

The following measures are used to judge the model:

Measure	Meaning
Accuracy	Share of all predictions that are correct.
Balanced accuracy	Average of the recall of the two classes. More useful when the classes are of different sizes.
ROC-AUC	How well the model ranks seizure examples above non-seizure examples over all thresholds.
PR-AUC	Summary of the precision-recall trade-off, which is helpful when the positive class is small.
Precision	Of the examples predicted as seizure, the share that really are seizures.
Recall	Of the real seizure examples, the share that the model detects.

3.4 RTL testbench

The testbench feeds in a sequence of input samples and compares the actual output with the expected label each time. Samples 1 to 20 are checked, every one matches, and the log ends with "Testbench finished".

This tells us the displayed test vectors pass. It does not cover every possible input, and it is not a clinical validation.

3.5 Verification steps

- Model validation: check the reported metrics and the errors for each class.
- Confusion matrix: separate the true negatives, false positives, false negatives and true positives.
- Schematic inspection: confirm that synthesis produced a logic network.
- Waveform inspection: observe the signals and the output during simulation.
- Test-vector checks: compare expected and actual outputs for each case.
- Power report: note the values shown, keeping in mind that they depend on constraints, switching activity and the target device.

4. Hardware-Oriented Implementation

The hardware part is written in Verilog RTL and examined through a synthesis schematic and simulation results. RTL describes behaviour at the register-transfer level, including combinational decision logic and any clocked state, and synthesis then maps it onto the logic resources of a device.

A schematic helps to see how the tool connected the logic, but it does not show that timing is met, that pins are constrained correctly, or that the design works on a real board.

4.1 RTL schematic

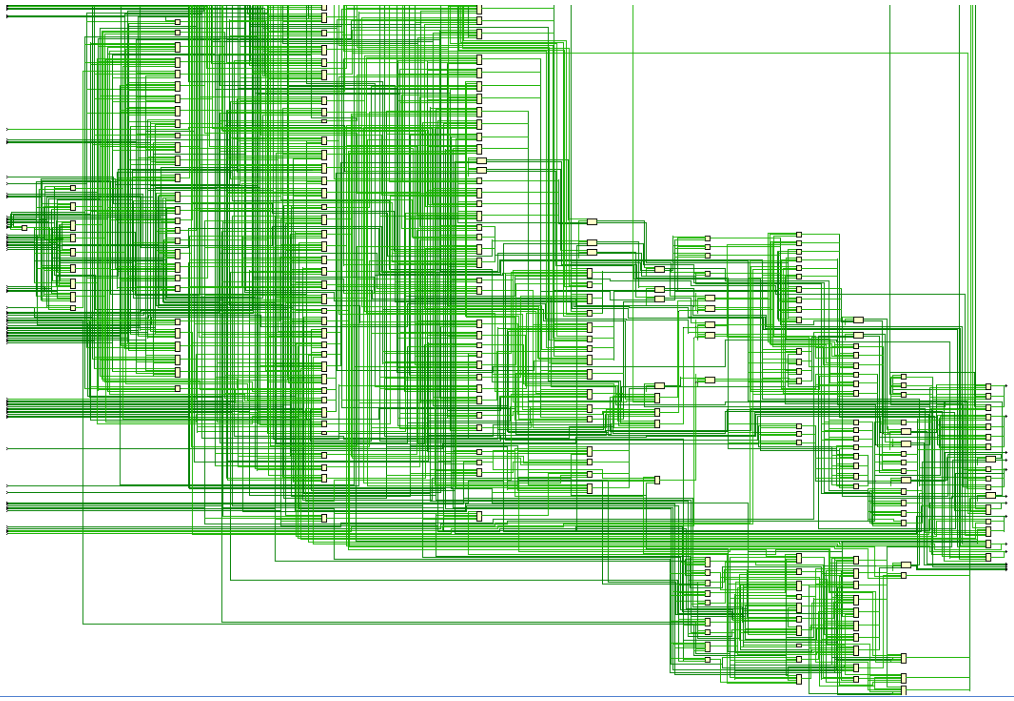


Figure 1. Synthesized RTL schematic.

The schematic is large and many signal labels are small at this size. For exact names of the top-level inputs, clock, reset and the prediction output, refer to the original design tool.

4.2 Simulation waveform

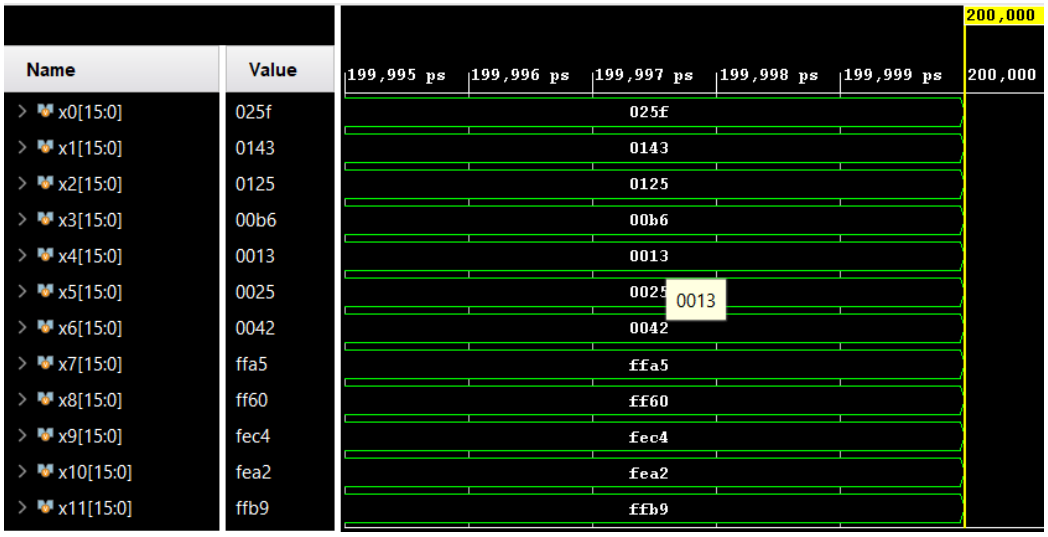


Figure 2. RTL simulation waveform.

The waveform shows a group of signals and their values over a short simulation interval. The signal names and time scale are only partly readable, so we do not quote a latency or clock-cycle count. It is best read together with the RTL source and the testbench.

4.3 Power estimation

The power estimate from the tool is summarised below.

Component	Power (W)	Share
Signals	3.851	25%
Logic	8.873	58%
I/O	2.465	17%
Dynamic power (total)	15.189	-
Device static power	0.246	-
Total on-chip power	15.435	-

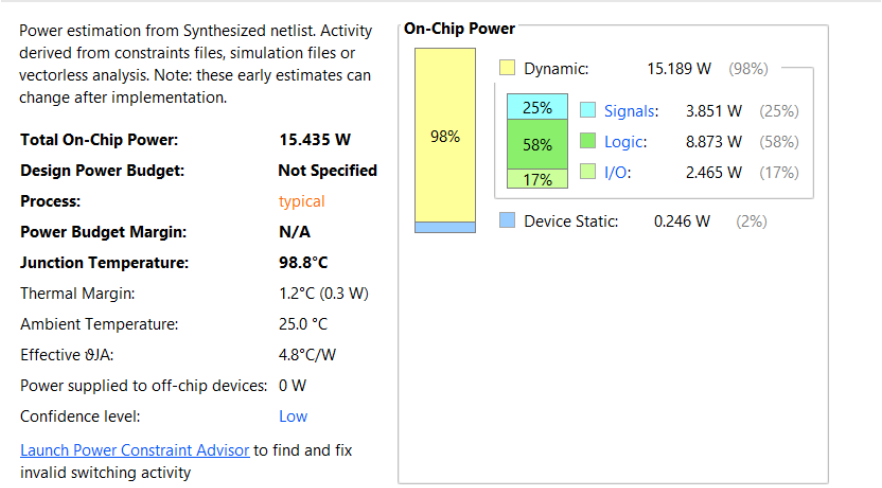


Figure 3. Power-estimation report.

These are estimated values, not measured power. The report marks the power budget as not specified and the confidence level as low. A total of 15.435 W also looks high for a design of this size, so it should be checked against the original tool report, the FPGA part, the constraints and the switching activity before being quoted as a final result.

5. Results and Discussion

5.1 Random Forest configuration

Parameter	Value
Training time	0.20 seconds
Number of trees	25
Maximum tree depth	3
Selected input features	16
Classification threshold	About 0.375
Exported model file	/content/random_forest_export.json

The training time was recorded in the environment used for training, so it is not the inference time of a hardware design. The export path also belongs to that environment.

```
Training time: 0.20 seconds
Number of trees: 25
Maximum tree depth: 3
```

Figure 4. Random Forest training configuration.

```
Trees exported: 25
Classification threshold: 0.3749999999999999
Selected input features: 16
Export saved to /content/random_forest_export.json
```

Figure 5. Exported model parameters.

5.2 Validation metrics

Metric	Value	Comment
Validation prediction time	0.0137 s	Depends on the environment used.
Accuracy	0.7875 (78.75%)	About 78.75% of predictions were correct.
Balanced accuracy	0.7690 (76.90%)	Average recall over the two classes.
ROC-AUC	0.8618	Good separation of the classes across thresholds.
PR-AUC	0.5416	Precision-recall performance for the seizure class.

A ROC-AUC of about 0.86 means the model separates the two classes reasonably well. But seizure is the minority class and its precision is only 0.35, so many seizure predictions turn out to be false alarms. A

recall of 0.74 means most seizure examples are caught, though some are still missed. For seizure detection this balance matters more than overall accuracy.

```
Validation prediction time: 0.0137 seconds
Validation accuracy: 0.7875370919881306
Validation balanced accuracy: 0.769031798766018
Validation ROC-AUC: 0.8618082581869959
Validation PR-AUC: 0.5416096768654648

Validation classification report:
              precision    recall  f1-score   support

 Non-seizure       0.95      0.79      0.87      1470
     Seizure       0.35      0.74      0.47       215

    accuracy                           0.79      1685
   macro avg       0.65      0.77      0.67      1685
weighted avg       0.88      0.79      0.82      1685

Validation confusion matrix:
[[1167  303]
 [  55  160]]
```

Figure 6. Validation metrics, classification report and confusion matrix.

5.3 Classification report

Class	Precision	Recall	F1-score	Support
Non-seizure	0.95	0.79	0.87	1,470
Seizure	0.35	0.74	0.47	215
Macro average	0.65	0.77	0.67	1,685
Weighted average	0.88	0.79	0.82	1,685
Overall accuracy	-	-	0.79	1,685

For non-seizure examples the precision is high (0.95), so a non-seizure prediction is usually right, and the recall is 0.79. For seizure examples the recall is 0.74, but the precision is 0.35 and the F1-score is 0.47.

This suggests the model leans towards catching seizures at the cost of many false positives. In a real system the threshold would need adjusting once it is decided how many missed seizures and how many false alarms are acceptable.

5.4 Confusion matrix

Actual class	Predicted non-seizure	Predicted seizure
Non-seizure	1,167 (true negatives)	303 (false positives)
Seizure	55 (false negatives)	160 (true positives)

Of the 1,470 non-seizure examples, 1,167 were classified correctly and 303 were wrongly flagged as seizure. Of the 215 seizure examples, 160 were detected and 55 were missed.

The false negatives are the most serious errors, since these are seizures the model did not detect. The false positives matter too, because they cause unnecessary alerts and extra checking.

5.5 RTL test cases

The testbench output shows 20 test cases, each comparing the expected result with the actual result. All 20 match.

Sample	Expected	Actual	Result
1	1	1	PASS
2	0	0	PASS
3	1	1	PASS
4	0	0	PASS
5	1	1	PASS
6	1	1	PASS
7	0	0	PASS
8	1	1	PASS
9	1	1	PASS
10	0	0	PASS

Sample	Expected	Actual	Result
11	1	1	PASS
12	1	1	PASS
13	1	1	PASS
14	1	1	PASS
15	1	1	PASS
16	1	1	PASS
17	1	1	PASS
18	1	1	PASS
19	1	1	PASS
20	1	1	PASS

```
Sample 1: expected=1, actual=1
Sample 2: expected=0, actual=0
Sample 3: expected=1, actual=1
Sample 4: expected=0, actual=0
Sample 5: expected=1, actual=1
Sample 6: expected=1, actual=1
Sample 7: expected=0, actual=0
Sample 8: expected=1, actual=1
Sample 9: expected=1, actual=1
Sample 10: expected=0, actual=0
Sample 11: expected=1, actual=1
Sample 12: expected=1, actual=1
Sample 13: expected=1, actual=1
Sample 14: expected=1, actual=1
Sample 15: expected=1, actual=1
Sample 16: expected=1, actual=1
Sample 17: expected=1, actual=1
Sample 18: expected=1, actual=1
Sample 19: expected=1, actual=1
Sample 20: expected=1, actual=1
Testbench finished.
```

Figure 7. Testbench console output.

The simulation ran to the end of the displayed sequence. We do not have the full test-vector source, assertion coverage, random testing or an exhaustive input check, so the test is limited to these 20 cases.

5.6 Synthesis and implementation

The presentation has a synthesized schematic and a power report, but no readable resource-utilisation or timing-summary table. So we do not report LUT, flip-flop, DSP or I/O counts, clock frequency or slack values. These should be taken from the original synthesis and implementation reports rather than guessed from the schematic.

6. Timing Analysis

A proper timing analysis needs the target FPGA part, the clock constraints, the input and output delay constraints and the timing summary from the implementation run.

The presentation has no readable timing summary showing worst negative slack, total negative slack, failing endpoints or the clock period, so we do not claim that timing constraints are met. The original Vivado timing summary should be added here once it is available.

7. Limitations

- The dataset name, how the data was split between subjects, the EEG acquisition details, the preprocessing and the feature names are not given in the presentation.
- The validation set is imbalanced (1,470 non-seizure against 215 seizure examples), so overall accuracy can hide weaknesses in one class.
- Seizure precision is 0.35, so there are many false positives at the chosen threshold.
- The metrics come from the validation set only, so they say nothing about unseen patients, other hospitals, other devices or other recording conditions.
- It is not shown that the full Random Forest was converted into equivalent Verilog logic.
- Only 20 RTL test cases are shown, and boundary cases and randomised testing are not documented.
- The power values are low-confidence estimates and need to be checked against the original tool report.
- Deployment on a physical FPGA and live EEG acquisition from electrodes have not been done.

This project is a technical prototype and must not be used for clinical diagnosis or treatment decisions.

8. Future Scope

- Document the EEG dataset, sampling rate, channels, preprocessing, feature extraction, labels and the subject-wise train, validation and test split.
- Evaluate on a held-out test set and, where possible, on recordings from subjects not used in training.
- Tune the threshold against a clear objective that balances seizure sensitivity and false alarms.
- Report sensitivity, specificity, precision, recall, F1-score, ROC-AUC and PR-AUC with confidence intervals, using repeated splits or cross-validation.
- Export a complete model specification: feature order, scaling, tree structures, threshold meaning and output encoding.
- Check whether the RTL implements the same model or a simplified decision block, and compare its predictions with the software model on the same inputs.
- Extend the testbench with boundary cases, random vectors, long sequences, reset behaviour and assertions.
- Re-run synthesis and implementation on the intended FPGA part with proper clock and I/O constraints, and record resource usage and post-route timing.
- Repeat the power estimation with realistic switching activity and the correct device settings.

- Once the technical side is validated, look at an EEG acquisition front end and a real-time data interface. Any medical use would also need clinical and regulatory validation.

9. Conclusion

This project combined Random Forest classification of EEG seizure data with a Verilog RTL simulation and a look at the hardware side. On the validation set the model reached 78.75% accuracy, 76.90% balanced accuracy, a ROC-AUC of 0.8618 and a PR-AUC of 0.5416.

For the seizure class the recall was 0.74 and the precision 0.35, so the model detects many seizure examples but also produces a fair number of false alarms. The model uses 25 trees, a maximum depth of 3, 16 features and a threshold of about 0.375. In the RTL simulation, all 20 displayed test cases matched their expected outputs.

Overall, this is a preliminary proof of concept for machine-learning classification alongside an RTL implementation. Better dataset documentation, testing on independent data, more thorough RTL verification and confirmed FPGA timing and power reports are needed before making any claim about real-time or clinical use.

References

1. Project presentation (cad_ppt.pptx): RTL schematic, power-estimation screenshot, simulation waveform, testbench cases and Random Forest results.
2. Breiman, L. "Random Forests." Machine Learning, 45, 5-32, 2001. https://doi.org/10.1023/A:1010933404324
3. AMD, Vivado Design Suite documentation (synthesis, simulation, timing analysis and power estimation).
4. Verilog HDL language reference and the project RTL and testbench source files.